

BARAKAZ

Admin Manual

A comprehensive reference for every feature of the Barakaz multivendor marketplace admin panel.

Version 1.0

Generated 26 April 2026

Confidential — Internal use

Contents

Contents	2
How to read this manual	4
Glossary	4
Part 1 — Getting Around the Admin Panel	6
1.1 Signing in	6
1.2 The admin shell	6
Sidebar groups	6
Header	7
1.3 Permission model in plain language	7
1.4 Mobile vs desktop	7
Part 2 — Feature Reference	8
2.1 Dashboard	8
2.2 Orders	9
2.3 Evidence Vault	9
2.4 Messages	10
2.5 Notifications	11
2.6 Products	11
2.7 Categories	12
2.8 Bulk Import	13
2.9 Media Library	13
2.10 Withdrawals	14
2.11 Subscriptions	14
2.12 Analytics	15
2.13 Marketing	15
2.13.1 Hero Banners	15
2.13.2 Promo Strip	16
2.14 Barakaz Academy	16
2.14.1 Lessons	16
2.14.2 Categories	17

2.15 Vendors	17
2.16 Users	17
2.17 Team & Roles	18
2.18 Settings	19
Part 3 — Cross-Cutting Workflows	20
3.1 Approve a new vendor (end-to-end)	20
3.2 Resolve a dispute (end-to-end)	20
3.3 Run a flash sale	20
3.4 Onboard a new staff member	21
3.5 Process a withdrawal request	21
Part 4 — Technical Appendix	22
4.1 Architecture overview	22
4.2 Permissions reference	22
4.3 Data model	23
Identity & access	23
Catalog	24
Commerce	24
Communications	24
Marketing & content	24
Compliance, safety, ops	25
4.4 Edge functions	25
4.5 Realtime subscriptions	26
4.6 Security model	26
4.7 Storage buckets	27
Quick reference — sidebar map	27
Quick reference — permission keys	27

Comprehensive reference for the Barakaz multivendor marketplace admin panel.

Version 1.0 — Generated 2026-04-26 Audience: Admin staff (Part 1–3) and technical operators (Part 4)

How to read this manual

This manual is split into four parts:

- **Part 1 — Getting around** explains how to sign in, the layout of the admin panel, and the permission model that decides what each staff member can see.
- **Part 2 — Feature reference** has one chapter per item in the admin sidebar. Every chapter follows the same template: purpose, how to access, page anatomy, step-by-step workflows, filters & search, edge cases, required permission, and the underlying database tables.
- **Part 3 — Cross-cutting workflows** walks through the end-to-end paths that touch multiple pages (approving a vendor, resolving a dispute, running a flash sale, etc.).
- **Part 4 — Technical appendix** documents the architecture, permissions reference, full data model, edge functions, realtime, and security model for engineers and operators.

Non-technical readers can stop after Part 3. Permission keys are written in `code style` and map 1-to-1 to the keys defined in `src/lib/permissions.ts`.

Glossary

- **Vendor** — A merchant store on Barakaz. Stored in the `vendors` table and linked to a single `auth.users` account via `user_id`.
- **Customer** — A buyer with a `customer` role in `user_roles`. Default role for any new sign-up.
- **Admin** — A staff member with the `admin` role in `user_roles`. Required to access `/admin`.
- **Super Admin** — An admin who is *not* assigned to any `team_role`. Has the wildcard permission `*` and sees every section.
- **Team role** — A named set of UI permission keys (e.g. "Logistics Manager", "Marketing Manager") stored in `team_roles.permissions` and assigned to admins via `team_members`.
- **Permission key** — A short string like `orders.view` that gates a sidebar item or button. Defined in `src/lib/permissions.ts`.
- **RLS** — Row-Level Security. Postgres rules attached to each table that decide which rows the current user can read or write.
- **Edge function** — Server-side TypeScript function deployed to the Lovable Cloud runtime. Used for order creation, email delivery, and analytics refresh.
- **Order** — A single checkout (`orders` row) that may contain items from many vendors. Each `order_items` row belongs to one vendor.
- **Dispute** — A formal complaint opened against an order (`disputes` row). While open, it freezes chat updates on that order.

- **Evidence Vault** — The admin view that combines an order, its dispute (if any), the chat history, attachments, and audit log into a single screen.
 - **Hero banner** — A full-width carousel slide on the homepage (`hero_banners`).
 - **Promotion** — A smaller card in the homepage promo strip (`promotions`).
 - **Resource** (Barakaz Academy) — A vendor-facing lesson, video, or article (`resources`).
-

Part 2 — Feature Reference

Each chapter follows this template:

Purpose · Access · Page anatomy · Workflows · Filters & search · Edge cases · Required permission · Underlying tables

2.1 Dashboard

Purpose. A single landing screen that shows marketplace health and surfaces work that needs immediate attention.

Access. `/admin` . Permission: `dashboard.view` .

Page anatomy (`AdminDashboard.tsx` + components):

- **Welcome row** — your name and the marketplace name pulled from `platform_settings` .
- **Performance KPI strip** — four stat cards (`AdminStatCard`): Revenue (rolling 30d), Orders today, New vendors awaiting approval, Open disputes. Each card shows the absolute number, the change vs the previous period, and a sparkline.
- **Action Required panel** (`ActionRequiredPanel.tsx`) — a prioritised queue of items needing a human: pending vendor approvals, pending withdrawal requests, pending subscription verifications, and open disputes. Each row links directly to the relevant page.
- **Quick Nav grid** (`QuickNavGrid.tsx`) — 8 large buttons that mirror the sidebar's most-used destinations (Orders, Vendors, Products, Withdrawals, Marketing, Academy, Team, Settings).
- **Recent Activity feed** (`RecentActivityFeed.tsx`) — last ~25 events: new orders, new vendor sign-ups, dispute openings, withdrawal requests. Updates in near-real-time via Supabase Realtime channels on `orders` , `vendors` , `disputes` , and `withdrawal_requests` .
- **Site Analytics widget** (`SiteAnalytics.tsx`) — chart of pageviews and unique visitors over the last 14 days, sourced from `site_analytics_cache` (refreshed by the `refresh-site-analytics` edge function).

Workflows.

- *Triage first thing in the morning.* Scan the four KPI cards, then work top-down through Action Required.
- *Spot a broken trend.* If the Revenue card shows a sharp dip, click through to Analytics for the breakdown.

Edge cases.

- New marketplaces with no data show "No activity yet" empty states inside each panel rather than blank cards.
- The analytics widget falls back to a "Last refreshed N hours ago" notice if the cache is stale (>6 h).

Underlying tables. `orders` , `order_items` , `vendors` , `disputes` , `withdrawal_requests` , `vendor_subscriptions` , `site_analytics_cache` , `notifications` .

2.2 Orders

Purpose. Inspect and manage every order placed across the marketplace, regardless of vendor.

Access. `/admin/orders` . Permissions: `orders.view` (read), `orders.update` (status changes).

Page anatomy (`AdminOrders.tsx`):

- **Page header** — title "All Orders", live order count.
- **Filter bar** — free-text filter on order ID or status; debounced.
- **Order table** — columns: Order ID (short), Customer name, Item count, Total, Payment status, Fulfilment status, Placed at, Actions.
- **Status pills** — colour-coded for `pending` , `processing` , `shipped` , `delivered` , `completed` , `cancelled` , `refunded` .
- **Row click** — opens the order detail drawer with the full shipping address, line items (each with vendor, variant, quantity, price, commission), payment method, and an action menu to change status. A status change is also written to `audit_logs` .

Workflows.

- *Find a customer's order.* Type any portion of the order ID into the filter; the table updates in place.
- *Cancel a stuck order.* Open the row → Actions → "Mark cancelled". This sets `orders.status = 'cancelled'` and emits a notification to the buyer.
- *Investigate a payment.* Read `payment_status` and `payment_method` , then jump to Evidence Vault for the chat thread.

Edge cases.

- Orders are inserted only by the `create-order` edge function (RLS rejects direct inserts), so the table is read-mostly. Status transitions go through `UPDATE` s with admin role.
- Vendors update *their* `order_items` independently (e.g. marking shipped); the parent `orders.status` rolls up to the most-blocking item state.

Underlying tables. `orders` , `order_items` , `addresses` , `audit_logs` , `notifications` .

2.3 Evidence Vault

Purpose. A purpose-built workspace for resolving buyer–vendor disputes. Combines the order, the chat thread, attachments, audit log, and any risk flags into a single read-and-act screen.

Access. `/admin/evidence` . Permission: `evidence.view` .

Page anatomy (`AdminEvidence.tsx`):

- **Search bar** — match by order ID, buyer name, buyer phone, or vendor store name.
- **Status filter** — All · Open · Closed · Escalated.
- **Disputes list** (left) — each row shows opened-at, order ID, buyer, vendor, reason snippet, current status pill.

- **Detail pane** (right) — selecting a dispute reveals four stacked sections:
 1. *Order summary* — items, totals, shipping address, payment status.
 2. *Dispute info* — `reason` , `opened_by` (buyer or vendor), opened/closed timestamps, `admin_notes` editor.
 3. *Chat snapshot* — full message timeline including system messages, edits, and soft-deleted messages flagged "(deleted)".
 4. *Attachments* — every file from `chat_attachments` for this order, viewable inline.
 5. *Audit log* — chronological list from `audit_logs` filtered to this `order_id` .
 6. *Risk flags* — any `user_risk_flags` rows for the buyer (e.g. excessive refunds, chargebacks).
- **Action toolbar.**
 - *Open dispute* (only when none exists) — prompts for a reason, inserts into `disputes` .
 - *Close dispute* — sets `disputes.status = 'closed'` and `closed_at = now()` . Re-enables chat updates on the order.
 - *Escalate* — sets `status = 'escalated'` , alerts senior admins via notifications.
 - *Save admin notes* — writes to `disputes.admin_notes` (visible only to admins).
 - *Add risk flag* — opens a small dialog to record `flag_type` , `score` , `reason` , optional `expires_at` .

Workflows. See §3.2 for the end-to-end dispute resolution path.

Edge cases.

- While `disputes.status = 'open'` , both buyer and vendor are blocked from soft-deleting or editing messages on that order (enforced in the `chat_messages` UPDATE policy via `order_has_open_dispute(order_id)`).
- Closing a dispute does not delete it — the row remains for the audit trail.

Underlying tables. `disputes` , `orders` , `order_items` , `chat_messages` , `chat_attachments` , `message_edit_history` , `audit_logs` , `user_risk_flags` , `vendors` , `profiles` .

2.4 Messages

Purpose. Read every conversation on the marketplace (buyer ↔ vendor) for moderation, training, and incident review.

Access. `/admin/messages` . Permission: `messages.view` .

Page anatomy (`AdminMessages.tsx`):

- **Conversation list** (left) — grouped by `conversation_id` and `vendor_id` , sorted by latest message. Each row shows the buyer name, vendor store, last-message preview, unread count, and a badge if any message has been edited or soft-deleted.
- **Thread pane** (right) — full chronological thread. Each bubble shows sender (Buyer / Vendor / System), timestamp, "edited" tag (with hover to view `message_edit_history`), and any attachment preview.

Soft-deleted messages render greyed-out with the original text restored from `original_message` .

- **Quick actions.** Open the order in Evidence Vault, copy a permalink to a specific message, view the buyer or vendor profile.

Edge cases.

- Messages tied to an order with an open dispute are read-only for participants; admins can still inspect history.
- System messages (`is_system_message = true`) are inserts made by the platform, e.g. "Order shipped" — they cannot be edited or deleted by anyone.

Underlying tables. `chat_messages` , `chat_attachments` , `message_edit_history` .

2.5 Notifications

Purpose. Send in-app notifications to one user, one vendor's owner, or every user.

Access. `/admin/notifications` . Permission: `notifications.send` .

Page anatomy (`AdminNotifications.tsx`):

- **Composer.**
- *Title* (≤80 chars), *Message* (textarea), *Type* (`info` , `success` , `warning` , `error`).
- *Audience* radio: All users · Specific vendor (autocomplete on store name) · Specific user (autocomplete on profile name).
- *Send* button — inserts one row per recipient into `notifications` .
- **Recent broadcasts** — the last ~50 notifications you have sent, with delivered count and read-rate.

Workflows.

- *Announce maintenance.* Audience = All, Type = warning, schedule the send during low traffic.
- *Nudge one vendor.* Audience = Specific vendor, write a personal message; the vendor sees the bell icon light up immediately (Realtime).

Edge cases.

- "All users" expands server-side; very large sends should be done off-peak.
- Notifications cannot be unsent — they can only be marked read by recipients.

Underlying tables. `notifications` .

2.6 Products

Purpose. Search, edit, feature, and moderate every product across the marketplace.

Access. `/admin/products` . Permissions: `products.view` (read), `products.update` (modify).

Page anatomy (`AdminProducts.tsx`):

- **Search bar** — fuzzy match on product name.

- **Product table** — columns: Image thumb, Name, Vendor, Category, Price, Stock, Status pill (`draft` · `active` · `inactive`), Featured flag, Actions.
- **Inline actions per row.**
- *Edit images & video* — opens an image manager dialog. Reorder, replace, delete images and paste a YouTube/Vimeo URL into `video_url` .
- *Set deal timer* — opens a small form to set `compare_at_price` and `deal_ends_at` . The countdown shows on the product card, search page, and homepage Today's Deals carousel.
- *Feature / Unfeature* — toggles `featured` (used by the homepage Featured strip).
- *Activate / Deactivate* — toggles `status` between `active` and `inactive` . Inactive products disappear from the storefront but remain in the vendor dashboard.

Edge cases.

- Stock values come from `product_variants.stock` summed when variants exist; otherwise from `products.stock` .
- Deleting a product is allowed only by its vendor (RLS), not by admins. To remove a product from the storefront, deactivate it.
- Setting `deal_ends_at` in the past will hide the deal timer immediately on the next page load.

Underlying tables. `products` , `product_images` , `product_variants` .

2.7 Categories

Purpose. Curate the storefront's category tree.

Access. `/admin/categories` . Permission: `categories.manage` .

Page anatomy (`AdminCategories.tsx`):

- **Tree view** — hierarchical list, drag-handle to reorder (display order). Each node shows its image thumb, name, slug, and child count.
- **Create / edit form.**
- *Name* — auto-fills *Slug* (lower-case, hyphenated). Slug is editable for legacy URLs.
- *Parent* — null for top-level, or pick another category for nesting.
- *Image* — uploaded to storage; used on category cards.
- **Delete** — only allowed when the category has no products and no children (enforced in the RLS-friendly delete handler).

Edge cases.

- Slugs must be unique. The form validates locally before insert; the database has a unique index as a backstop.
- Renaming a category does *not* change its slug automatically — preserves SEO.

Underlying tables. `categories` , `products` (for product counts).

2.8 Bulk Import

Purpose. Import many products at once from a spreadsheet, on behalf of any vendor.

Access. `/admin/bulk-import` . Permission: `bulk_import.use` .

Page anatomy (`AdminBulkImport.tsx` → `BulkProductImport`):

- **Vendor selector** — pick the destination vendor (only approved vendors are listed). Required.
- **Template downloads** — buttons to download a CSV or Excel template with the canonical column headers and one example row.
- **Upload area** — drag-and-drop or browse. Accepts `.csv` , `.xlsx` . Reads the file in the browser.
- **Validation pass.** Each row is checked for: required fields, valid category slug, numeric price/stock, valid status, image URLs reachable.
- **Dry-run preview** — table of parsed rows with per-row status (☐ ready / ☐ error). Errors show the column and the message.
- **Import button** — runs the insert in batches; shows a progress bar and a final summary (created · skipped · failed).
- **Error CSV download** — only the failed rows, with an extra "error" column, so you can fix and re-upload.

Edge cases.

- Categories are matched by slug. A missing category fails the row; the admin must create it first in §2.7.
- `key_features` and `whats_in_box` columns accept a `;` -separated list.
- Product variants are *not* part of bulk import — create them in the vendor product editor.

Underlying tables. `products` , `product_images` , `vendors` .

2.9 Media Library

Purpose. Cross-vendor view of every uploaded image, with admin-level delete and short-link generation.

Access. `/admin/media` . Permission: `media.manage` .

Page anatomy (`AdminMedia.tsx` → `MediaLibrary mode="admin"`):

- **Filters** — by vendor, by linked product, by upload date.
- **Grid view** — image tiles with hover overlay showing dimensions, file size, vendor, and the product it belongs to.
- **Per-image actions.**
- *View full size* — opens in a lightbox.
- *Copy short link* — generates a `short_links` row and copies a tidy URL like `https://barakaz.com/s/abc123` .
- *Replace* — upload a new file that keeps the same URL slot.
- *Delete* — removes the file from storage and the `product_images` row.

Edge cases.

- Deleting an image used as a product's only image will leave the product without a thumbnail; the storefront falls back to the placeholder.
- Short links are publicly readable but only owners or admins can update / delete them.

Underlying tables. `product_images` , `short_links` , plus storage buckets.

2.10 Withdrawals

Purpose. Approve or reject vendor payout requests.

Access. `/admin/withdrawals` . Permissions: `withdrawals.view` (read), `withdrawals.update` (decide).

Page anatomy (`AdminWithdrawals.tsx`):

- **Request queue** — table of pending withdrawal requests with vendor name, requested amount, available balance at request time, payment method (M-Pesa, bank), and submitted date.
- **Per-row action.**
- *Approve* — opens a confirmation with an optional admin note. Marks the request approved, debits the vendor's available balance, and notifies the vendor.
- *Reject* — requires a note (visible to the vendor as the rejection reason).
- **History tab** — all decisions with the deciding admin, decision time, and note. Filterable.

Edge cases.

- The "available balance" snapshot is taken at request time; if a refund happens between request and approval, the admin sees a warning banner before approving.
- Approving does *not* push money to a payment rail automatically — payout is operational.

Underlying tables. `withdrawal_requests` , `vendors` , `notifications` , `audit_logs` .

2.11 Subscriptions

Purpose. Manage vendor subscription plans and verify manual payment receipts.

Access. `/admin/subscriptions` . Permissions: `subscriptions.view` (read), `subscriptions.update` (decide).

Page anatomy (`AdminSubscriptions.tsx`):

- **Pending verifications** (top section) — vendors who paid via M-Pesa and submitted a transaction code. Columns: vendor, plan, paid amount expected, transaction code, submitted at. Actions: *Confirm payment* or *Reject* with reason.
- **All subscriptions** (bottom section) — every vendor and their current plan, expiry, and renewal status. Search by vendor name. Actions: change plan, extend expiry, cancel.
- **Plan tiers** are defined in `src/lib/subscriptionPlans.ts` . They drive the price and the included features (product limits, featured slots, etc.).

Edge cases.

- Confirming a payment activates the plan and resets the expiry to `now() + plan.duration`.
- Rejecting requires a note; the vendor sees a notification and is prompted to retry.

Underlying tables. `vendor_subscriptions`, `vendors`, `notifications`.

2.12 Analytics

Purpose. Marketplace-wide traffic and conversion analytics.

Access. `/admin/analytics`. Permission: `analytics.view`.

Page anatomy (`AdminAnalyticsPage.tsx`):

- **Date range picker** — last 7, 30, 90 days or custom.
- **Headline KPIs** — Sessions, Unique visitors, Add-to-cart rate, Conversion rate, Average order value.
- **Trend charts** — sessions and orders over time, stacked by traffic source where available.
- **Top products** — by views, by orders, by revenue.
- **Top vendors** — by revenue.
- **Refresh notice** — shows "Last refreshed at HH:MM" pulled from `site_analytics_cache.updated_at`. The `refresh-site-analytics` edge function rebuilds the cache on a schedule (and can be triggered manually from this page).

Edge cases.

- If the cache is empty (cold start) the page shows a "Building first snapshot" banner and triggers the refresh function.
- Visitor counts use deduplicated session IDs to bypass Edge Function row limits.

Underlying tables. `site_analytics_cache`, `vendor_analytics`.

2.13 Marketing

Purpose. Run the storefront's hero carousel and promo strips without engineering.

Access. `/admin/marketing`. Permission: `marketing.manage`.

Page anatomy (`AdminMarketing.tsx`):

- **Tabs:** *Hero Banners* · *Promo Strip*.

2.13.1 Hero Banners

- **List** — each banner shows desktop preview, mobile preview, schedule window, status (Live / Scheduled / Expired / Off), and display order.
- **Create / edit form.**
- *Desktop image* (required) and *Mobile image* (recommended) — uploaded via the Media Library.
- *Title, Subtitle, CTA label* (e.g. "Shop now").

- *Link target* picker — choose a category, product, vendor store, or paste an arbitrary URL.
- *Display order* — lower = earlier in the carousel.
- *Schedule* — `starts_at` / `ends_at` . Empty = always.
- *Active toggle* — quick on/off without deleting.
- **Reorder** — drag the row handle to set `display_order` .

2.13.2 Promo Strip

- Smaller cards (brand or product). Same fields as hero banners plus a `placement` (top strip, mid strip, etc.) and `kind` (`brand` or `product`).

Edge cases.

- Banner visibility on the storefront is RLS-driven: `is_active` AND `now()` between `starts_at` and `ends_at` . Schedule windows take effect immediately.
- Deleting a banner removes it permanently; prefer toggling off.

Underlying tables. `hero_banners` , `promotions` .

2.14 Barakaz Academy

Purpose. Publish lessons, videos, and articles that help vendors succeed. Renamed from "Resource Center".

Access. `/admin/resources` . Permission: `resources.manage` .

Page anatomy (`AdminResources.tsx`):

- **Tabs:** *Lessons* · *Categories*.

2.14.1 Lessons

- **Search** by title.
- **Lesson table** — Title, Category, Type (`article` , `video`), Difficulty (`beginner` , `intermediate` , `advanced`), Duration (min), Published, Featured, View count.
- **Create / edit form.**
- *Title*, *Slug* (auto-generated, editable).
- *Category* — pick from `resource_categories` .
- *Summary* (max 300 chars) — appears on cards and previews.
- *Content* — long-form text body.
- *Resource type* — Article or Video.
- *Cover image* — uploaded or pasted URL.
- *Video* (if type = video) — provider (YouTube / Vimeo) + URL, or upload a file.
- *Gallery* — list of images with captions, drag to reorder.
- *Attachments* — JSON array of downloadable files (PDFs, templates).

- *Tags* — free-text, press Enter to add.
- *Difficulty*, *Duration (minutes)*.
- *Display order* — lower = earlier in the category page.
- *Publish* and *Feature* toggles.
- **View counts** — auto-incremented on every public view.
- **Feedback** — readers mark a lesson "helpful" or not (`resource_feedback`); aggregated counts show on the table.

2.14.2 Categories

- Add, rename, reorder, deactivate. Each category has an icon name (Lucide icon) and a colour.

Edge cases.

- Unpublished lessons are hidden from the public Academy but remain editable.
- Deleting a category requires moving its lessons first.

Underlying tables. `resources` , `resource_categories` , `resource_images` , `resource_progress` , `resource_feedback` .

2.15 Vendors

Purpose. Approve, suspend, and review every vendor account.

Access. `/admin/vendors` . Permissions: `vendors.view` , `vendors.update` .

Page anatomy (`AdminVendors.tsx`):

- **Search** by store name.
- **Vendor table** — Store name, Owner profile, Country, Created, Status pill (`pending` , `approved` , `suspended` , `rejected`), Subscription plan.
- **Per-row actions.**
- *Approve* (from pending) — flips status to `approved` and notifies the owner.
- *Suspend* — temporarily hides the vendor's products from the storefront; products remain in the vendor dashboard.
- *Reactivate* — undoes a suspension.
- *View store* — opens the public store page.
- *Edit* — opens the vendor profile drawer (store name, logo, description, contact, payout details).

Workflows. See §3.1 for the end-to-end approval path.

Underlying tables. `vendors` , `vendor_subscriptions` , `profiles` , `notifications` .

2.16 Users

Purpose. Browse profiles and manage role assignments.

Access. `/admin/users` . Permissions: `users.view` , `users.assign_roles` .

Page anatomy (`AdminUsers.tsx`):

- **Search** by full name (case-insensitive).
- **Profile table** — Avatar, Full name, Roles (pills), Joined date, Actions.
- **Per-row actions.**
- `+ Vendor` — adds the `vendor` role. Useful when manually onboarding a known seller.
- `+ Admin` — adds the `admin` role. Sensitive — use sparingly; the new admin defaults to Super Admin until you give them a team role in §2.17.

Edge cases.

- The action buttons hide once the role is already present.
- Removing a role is done from the database (intentional friction). Future versions will expose it here.

Underlying tables. `profiles` , `user_roles` .

2.17 Team & Roles

Purpose. Define which admin staff can see which sections of the admin panel.

Access. `/admin/team` . Permission: `team.manage` (UI gate). Visible only to Super Admins by design.

Page anatomy (`AdminTeam.tsx`):

- **Roles tab.**
- List of roles (system + custom). System roles are read-only.
- *Create role* — name, description, and a permission matrix grouped by sidebar section. Tick the keys you want to grant.
- *Edit role* — update name/description/permissions. System role permissions are locked.
- *Delete role* — only if no team members are using it.
- **Members tab.**
- List of admins and their assigned role.
- *Assign* — pick an admin (only those with the `admin` role appear) and a role. Inserts a `team_members` row.
- *Reassign* — update the `team_role`.
- *Remove* — deletes the membership; the admin reverts to Super Admin.

Workflows.

- *Logistics manager* — give Orders (view + update), Evidence Vault (view), Messages (view), Withdrawals (view).
- *Marketing manager* — give Marketing (manage), Academy (manage), Notifications (send).

- *Catalog editor* — give Products (view + update), Categories (manage), Bulk Import (use), Media (manage).

Edge cases.

- The full key list is in `PERMISSIONS` (see §4.2). Adding a new key in code automatically surfaces it in the matrix on next load.
- Deleting a role used by even one team member is blocked; reassign first.

Underlying tables. `team_roles` , `team_members` , `user_roles` .

2.18 Settings

Purpose. Platform-wide configuration: brand identity, payment instructions, and integrations.

Access. `/admin/settings` . Permission: `settings.manage` .

Page anatomy (`AdminSettings.tsx`):

- **M-Pesa section.**
- *Till number, Paybill, Account name, Phone, Instructions* (multi-line shown to buyers at checkout).
- **Brand & marketplace identity** — site name, support email, support phone, default country.
- **Order behaviour** — auto-cancel window for unpaid orders, default order status flow.
- **Email** — from-name and from-address shown in transactional emails.
- **Save** — persists to `platform_settings` (key/value JSON store).

Edge cases.

- Changes take effect immediately for new requests; cached pages may take a few minutes.
- Sensitive credentials (API keys) live in Supabase Vault, not in `platform_settings` .

Underlying tables. `platform_settings` .

Part 3 — Cross-Cutting Workflows

3.1 Approve a new vendor (end-to-end)

1. New vendor signs up at `/vendor/register`. A row is created in `vendors` with `status = 'pending'`.
2. The Dashboard's Action Required panel surfaces the new vendor.
3. Admin opens **Vendors** (§2.15), clicks the vendor's row to review the store profile, payout details, and any uploaded documents.
4. If acceptable, click *Approve*. This sets `vendors.status = 'approved'` and inserts a `notifications` row for the vendor's owner.
5. The vendor immediately gains access to the vendor dashboard and can list products. RLS on `products` keys off `vendor.user_id`.
6. (Optional) Open **Subscriptions** (§2.11) to assign or confirm the vendor's plan.

3.2 Resolve a dispute (end-to-end)

1. Either party files a dispute from the order page; a row appears in `disputes` with `status = 'open'`.
2. The Dashboard surfaces the open dispute. Admin clicks through to **Evidence Vault** (§2.3).
3. Read the order summary, the chat thread (including any deleted/edited messages), and any attachments.
4. Add `admin_notes` capturing your findings.
5. Decide:
 - *Refund the buyer* — change the order status to `refunded` from **Orders** (§2.2). Notify the vendor.
 - *Side with the vendor* — leave the order status as is.
 - *Escalate* — flip the dispute to `escalated` and tag a senior admin via Notifications (§2.5).
1. Click *Close dispute*. The chat unlocks; the audit log captures the close.
2. (Optional) Add a `user_risk_flag` if the buyer or vendor exhibits a pattern.

3.3 Run a flash sale

1. Open **Products** (§2.6), find the products in the sale, click *Set deal timer* on each. Set `compare_at_price` (the original price for the strikethrough) and `deal_ends_at`.
2. Open **Marketing** (§2.13) → Hero Banners. Create a banner with a "Flash Sale" image, link target = the search page filtered to deals (`/search?deals=1`), and a schedule that matches the sale window.
3. Optionally feature key products via Products → *Feature* so they appear in the homepage Featured strip.
4. Use **Notifications** (§2.5) to send an "All users" announcement at launch.

5. After the sale ends, the deal timer auto-expires and the banner deactivates per its schedule. No clean-up required.

3.4 Onboard a new staff member

1. Have them register normally at `/auth`.
2. Open **Users** (§2.16), search by name, click + *Admin*. They can now sign in to `/admin`.
3. Open **Team & Roles** (§2.17) → *Members* → *Assign*. Pick a team role (or create a new one in the *Roles* tab first). They now see only the sections their role allows.
4. (Optional) Have them sign out and back in to refresh permissions on the client.

3.5 Process a withdrawal request

1. The Dashboard surfaces pending withdrawals.
 2. Open **Withdrawals** (§2.10). Review the vendor's balance and recent activity.
 3. *Approve* (with optional note) or *Reject* (note required).
 4. Pay out via your operational rail (manual M-Pesa, bank transfer, etc.).
 5. Mark the request as paid (if your workflow uses the "paid" sub-status).
 6. The vendor receives a notification.
-

Part 4 — Technical Appendix

4.1 Architecture overview

- **Frontend.** React 18 + Vite 5 + TypeScript 5 + Tailwind CSS v3 + shadcn/ui. Routing via `react-router-dom`. Server state via `@tanstack/react-query`. Realtime via `supabase-js`.
- **Backend.** Lovable Cloud (Supabase under the hood). Postgres database with RLS, Auth (email + Google OAuth), Storage buckets, and Edge Functions on Deno.
- **AI.** Lovable AI Gateway is available (no key required) for any future AI-powered admin features.
- **Hosting.** Storefront published at `https://shopclave-pro.lovable.app` and the custom domains `https://www.barakaz.com` / `https://barakaz.com`.

The admin panel is a `/admin/*` route subtree inside the same SPA as the storefront, gated by `AdminLayout` and per-route `RequirePermission` wrappers.

4.2 Permissions reference

All keys are defined in `src/lib/permissions.ts`. The wildcard `*` grants every key (Super Admin).

Key	Sidebar item / use	Notes
<code>dashboard.view</code>	Dashboard	
<code>orders.view</code>	Orders (read)	
<code>orders.update</code>	Orders (status changes, cancellations)	
<code>products.view</code>	Products (read)	
<code>products.update</code>	Products (edit, feature, activate, deal timer)	
<code>categories.manage</code>	Categories	Create, edit, delete
<code>bulk_import.use</code>	Bulk Import	
<code>media.manage</code>	Media Library	
<code>vendors.view</code>	Vendors (read)	
<code>vendors.update</code>	Vendors (approve, suspend, edit profile)	
<code>users.view</code>	Users (read)	
<code>users.assign_roles</code>	Users (+ Vendor / + Admin buttons)	
<code>withdrawals.view</code>	Withdrawals (read)	

Key	Sidebar item / use	Notes
<code>withdrawals.update</code>	Withdrawals (approve, reject)	
<code>subscriptions.view</code>	Subscriptions (read)	
<code>subscriptions.update</code>	Subscriptions (confirm, reject, change plan)	
<code>analytics.view</code>	Analytics	
<code>messages.view</code>	Messages	
<code>evidence.view</code>	Evidence Vault	
<code>notifications.send</code>	Notifications composer	
<code>settings.manage</code>	Settings	
<code>team.manage</code>	Team & Roles	Granted by default to Super Admins; never granted to regular roles unless intentional
<code>marketing.manage</code>	Marketing (banners + promotions)	Also drives RLS on <code>hero_banners</code> and <code>promotions</code>
<code>resources.manage</code>	Barakaz Academy	Also drives RLS on <code>resources</code> and friends via <code>can_manage_resources()</code>

`hasPerm(perms, key)` — returns true if the user has `*` or the specific key. Used throughout the UI and inside three Postgres helpers: `has_role`, `has_permission`, `can_manage_resources`.

4.3 Data model

This section lists every table the admin panel reads or writes, plus its purpose, key columns, and a one-line RLS summary. All tables use UUID primary keys unless stated.

Identity & access

- `profiles` — display name, avatar, phone for each user. Linked to `auth.users.id` via `user_id`. *Read*: self or admin. *Write*: self only.
- `user_roles` — `(user_id, role)` rows. `role` enum: `customer`, `vendor`, `admin`. *Read*: self or admin. *Write*: admin only. Avoids the recursive RLS pitfall by being a separate table.
- `team_roles` — named bundles of UI permission keys (`permissions text[]`). *Manage*: `team.manage`. *Read*: admin.
- `team_members` — `(user_id, team_role_id)`. *Manage*: `team.manage`. *Read*: admin.
- `addresses` — buyer shipping addresses. *Read/write*: owner only.

Catalog

- `categories` — `name`, `slug`, `parent_id`, `image`. Public read. *Write*: admin only.
- `products` — `name`, `price`, `compare_at_price`, `deal_ends_at`, `featured`, `status`, `key_features` `text[]`, `whats_in_box` `text[]`, `condition`, `video_url`. Active products are public; vendors manage their own; admins read/update all.
- `product_variants` — `variant_options` `jsonb`, `stock`, `price`, `image_url`. Public read; vendor or admin manage.
- `product_images` — `position`, `url`, optional `variant_id`. Public read; vendor manages.
- `reviews` — only purchasers can insert (RLS checks the buyer has a `delivered` / `completed` order item for the product). Public read.

Commerce

- `orders` — `total`, `status`, `payment_status`, `payment_method`, `shipping_address` `jsonb`. Inserts allowed only by the `create-order` edge function (RLS `WITH CHECK false` for direct inserts). Owner reads own; admin reads all + updates all.
- `order_items` — child of orders. `vendor_id`, `quantity`, `price`, `commission_amount`, `status`, `variant_options` `jsonb`. Vendor reads/updates their own; admin reads all.
- `disputes` — (`order_id`, `status`, `reason`, `admin_notes`, `opened_by`, `opened_at`, `closed_at`). Admin manages all; buyer/vendor read their own order's dispute.
- `vendors` — store profile + payout details. `status`: pending/approved/suspended/rejected.
- `vendor_subscriptions` — current plan, expiry, payment proof for manual M-Pesa confirmations.
- `withdrawal_requests` — payout queue.
- `vendor_analytics` — append-only event log (`event_type`, `product_id`). Inserted by anyone; read by the vendor or admin.

Communications

- `chat_messages` — full message history with edit/delete metadata: `original_message`, `original_attachment_url`, `deleted_by_sender`, `deleted_by_receiver`, `is_system_message`. Update allowed only when no open dispute exists on the linked order.
- `chat_attachments` — file metadata for chat uploads. Read by participants and admins.
- `message_edit_history` — append-only history of message edits. Read by editor or admin.
- `notifications` — in-app notifications. Recipients read/update own; admins read all and broadcast.

Marketing & content

- `hero_banners` — homepage carousel. Public read of currently-live banners; `marketing.manage` for everything else.
- `promotions` — homepage promo strip; same access pattern as banners.

- `resources` + `resource_categories` + `resource_images` + `resource_progress` + `resource_feedback` — Academy content and reader engagement. Public read of published items; `resources.manage` for editing (helper: `can_manage_resources`).

Compliance, safety, ops

- `audit_logs` — admin-visible activity log. Insert from triggers/edge functions; read by admin only.
- `user_risk_flags` — admin-managed risk markers on users.
- `cookie_consent` — append-only consent log per session.
- `platform_settings` — key/value JSON config store. Admin-only.
- `site_analytics_cache` — pre-aggregated traffic snapshot used by the Dashboard widget and Analytics page.
- `short_links` — URL shortener for media and shareable links.
- `email_send_log` , `email_send_state` , `email_unsubscribe_tokens` , `suppressed_emails` — operational tables for the transactional email pipeline. Service-role only.

4.4 Edge functions

All functions are deployed automatically with the project. Logs are visible in the Cloud overview.

Function	Trigger	Purpose
<code>create-order</code>	Called from the storefront checkout	Validates the cart, computes per-item commission, inserts <code>orders</code> + <code>order_items</code> , returns the new order ID. Direct inserts to <code>orders</code> are blocked by RLS so this is the single entry point.
<code>auth-email-hook</code>	Supabase Auth webhook	Renders branded auth emails (signup, magic link, recovery, email change, reauthentication, invite) using the React Email templates in <code>_shared/email-templates/</code> .
<code>process-email-queue</code>	Cron (every minute or so)	Drains queued transactional emails respecting <code>email_send_state.batch_size</code> and rate limits.
<code>send-transactional-email</code>	Called by other functions / triggers	Sends a single transactional email by template name + payload. Logs to <code>email_send_log</code> .

Function	Trigger	Purpose
<code>preview-transactional-email</code>	Admin-only (debug)	Renders a transactional template to HTML for visual QA.
<code>handle-email-suppression</code>	Webhook from email provider	Records bounces / complaints into <code>suppressed_emails</code> so future sends skip them.
<code>handle-email-unsubscribe</code>	Public link from email	Validates the token and marks the address unsubscribed.
<code>refresh-site-analytics</code>	Cron + manual trigger from §2.12	Rebuilds <code>site_analytics_cache</code> from raw events.

4.5 Realtime subscriptions

The admin panel subscribes to Postgres Changes on:

- `orders` — Dashboard activity feed, Orders page list.
- `disputes` — Dashboard Action Required, Evidence Vault list.
- `notifications` — Header bell badge.
- `chat_messages` — Messages page (live updates as messages arrive).
- `withdrawal_requests` — Dashboard and Withdrawals page.

Tables that are *intentionally excluded* from public realtime broadcasts (per the project's privacy hardening) include `profiles`, `user_roles`, `addresses`, `audit_logs`, `email_send_log`. Admin pages still query them on demand.

4.6 Security model

- **Two-factor gate.** Admin role at the database level + permission keys at the UI level. UI gates fail closed.
- `has_role` is `SECURITY DEFINER`. Lookups against `user_roles` happen with elevated privileges to avoid recursive RLS.
- **Role table is separate.** Roles are never stored on `profiles` to prevent privilege-escalation attacks via profile updates.
- **Admin-only inserts on sensitive tables** are enforced by `has_role(auth.uid(), 'admin')` in the policy.
- **Order inserts are server-only.** The `orders` table refuses client inserts (`WITH CHECK false`) so all orders flow through `create-order`.
- **Audit trail.** `audit_logs` captures sensitive actions (status flips, dispute decisions, role changes). Read-only to admins.

- **Email safety.** Suppression list and unsubscribe tokens prevent re-sending to opted-out addresses.

4.7 Storage buckets

- **product-images** — vendor and admin uploads for product photos and gallery shots.
- **resource-media** — Barakaz Academy covers, gallery, attachments, and uploaded videos.
- **chat-attachments** — files attached to buyer-vendor messages.
- **brand-assets** — hero banners, promo strip artwork, category images.

All buckets use signed URLs for private files and public URLs (with short-link wrappers) for storefront images.

Quick reference — sidebar map

Overview	■ Dashboard
Operations	■ Orders · Evidence Vault · Messages · Notifications
Catalog	■ Products · Categories · Bulk Import · Media
Finance	■ Withdrawals · Subscriptions
Insights	■ Analytics
Content	■ Marketing · Barakaz Academy
System	■ Vendors · Users · Team & Roles · Settings

Quick reference — permission keys

dashboard.view		
orders.view	orders.update	
products.view	products.update	
categories.manage	bulk_import.use	media.manage
vendors.view	vendors.update	
users.view	users.assign_roles	
withdrawals.view	withdrawals.update	
subscriptions.view	subscriptions.update	
analytics.view		
messages.view	evidence.view	notifications.send
settings.manage	team.manage	
marketing.manage	resources.manage	
* (Super Admin: grants all of the above)		

— End of manual —